Ecualizador digital

Cherro Cristian, Fuster Lucio

Departamento de electrónica, Universidad Tecnológica Nacional FRA

Buenos Aires, Argentina

luciofuster@gmail.com

cristiancherro@hotmail.com.ar

***Abstract*— Este proyecto presenta el diseño e implementación de un ecualizador digital que procesa señales de audio mediante un microcontrolador ESP32 usando el sistema operativo en tiempo real FreeRTOS. El sistema divide la señal en tres bandas con ajuste de ganancia en cada una. Se integran módulos de adquisición analógica, procesamiento digital con algoritmos DSP y conversor para salida analógica.**

ÍNDICE

I. INTRODUCCIÓN……..1
II. OBJETIVOS……..1
III. DIAGRAMA EN BLOQUES……1
IV. HARDWARE……1
A. ADC I²S……...1
B. DAC I²S……...2
C. LCD 20x04……..……2
D. Encoder rotativo……..……..2
E. UART PC……..……2
F. ESP 32……..……3
VI. SOFTWARE……3
VII. CONCLUSIONES……4
REFERENCIAS……..……4
ANEXOS……..……5

I. INTRODUCCIÓN

El procesamiento digital de señales (DSP) aplicado al audio constituye una herramienta esencial en aplicaciones modernas de comunicación y entretenimiento. En este prototipo se desarrolla el diseño de un ecualizador digital implementado sobre un microcontrolador ESP32 [1] con soporte de FreeRTOS[2].

II. OBJETIVOS

Por consigna el objetivo es: diseñar un sistema de procesamiento digital de señales que permita filtrar una señal de audio en tiempo real dividiéndola en 3 bandas de frecuencia donde a cada una de esas bandas se le debe poder ajustar la ganancia en un rango de ±10 dB y donde se debe poder ver su configuración actual. Además debe contar con: persistencia en la configuración; elementos de HMI; gestión remota de comandos; y un PCB prototipo.

III. DIAGRAMA EN BLOQUES

El siguiente diagrama en bloques muestra las conexiones que tendrá el prototipo entre módulos, bloques funcionales y el microcontrolador.

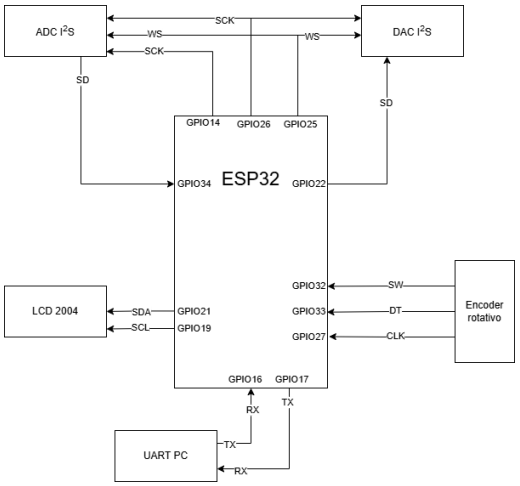


Fig. 1 diagrama en bloques

IV. HARDWARE

A. ADC I²S

El bloque funcional del ADC I²S estará construido en el PCB y está basado en el circuito integrado PCM1808 [3], es un ADC con protocolo de comunicación I²S con capacidad de funcionar en formato de 24 bits y frecuencias de muestreo que van de 8kHz a 96kHz y posibilidad de funcionar tanto como maestro o esclavo. En nuestro sistema funcionará como esclavo de la comunicación I²S, establecemos frecuencia de sampleo a 44.1kHz para muestrear la señal de entrada. Este se va a comunicar al microcontrolador mediante el hardware incluido I²S guardando los datos por DMA.



Fig. 2 integrado PCM1808

6.5 Electrical Characteristics

All specifications at T_A = 25°C, V_{CC} = 5 V, V_{DD} = 3.3 V, master mode, f_S = 48 kHz, system clock = 512 f_S, 24-bit data, unless otherwise noted

PARAMETER	TEST CONDITIONS	MIN	TYP	MAX	UNIT
Resolution			24		Bits
DATA FORMAT					
Audio data interface format			I²S, left-justified		
Audio data bit length			24		Bits
Audio data format			MSB-first, 2s complement		
f_S Sampling frequency		8	48	96	kHz
System clock frequency	256 f_S	2.048	12.288	24.576	MHz
	384 f_S	3.072	18.432	36.864	
	512 f_S	4.096	24.576	49.152	

Tabla 1 parámetros de funcionamiento PCM1808

B. DAC I²S

Para el DAC I²S se usará un módulo basado en el integrado PCM5102 [4], este integrado soporta resolución de palabra de 16, 24 y 32 bit, a fin de mantener la consistencia entre ADC y DAC usaremos el formato de 24 bit. También puede soportar frecuencias de muestreo que van de 8kHz a 384kHz, la cual nuevamente se configurará a 44.1kHz por consistencia entre dispositivos. Este al igual que el ADC funcionará como esclavo de la comunicación I²S y usará los datos procesados por DSP en el microcontrolador para generar a su salida una señal analógica. Según los parámetros que da su hoja de datos tiene como máximo una salida de 2.1 Vrms con una potencia máxima de salida de 6dBm (en condición de carga mínima de 1kΩ).

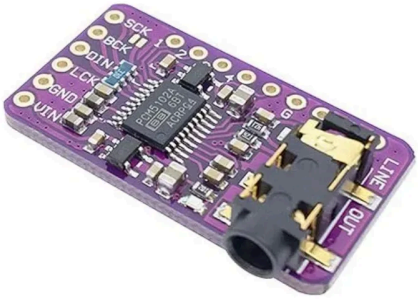


Fig. 3 módulo basado en PCM5102

PARAMETER		TEST CONDITIONS	MIN	TYP	MAX	UNIT
	Resolution		16	24	32	Bits
Data Format (PCM Mode)						
	Audio data interface format		I²S, left justified			
	Audio data bit length		16, 24, 32-bit acceptable			
	Audio data format		MSB First, 2's Complement			
f_S (1)	Sampling frequency		8		384	kHz
	System clock frequency		64, 128, 192, 256, 384, 512, 768, 1024, 1152, 1536, 2048, or 3072 f_{SCK}, up to 50Mhz			

Tabla 2 parámetros de funcionamiento PCM180

C. LCD 20x04

Como elemento visual de la HMI en el prototipo se usará un display LCD de 4 filas y 20 caracteres el cual indicará la configuración actual y también contará con un cursor que indica el parámetro que se quiere modificar actualmente. Este display cuenta con un módulo adaptador de comunicación por i²C basado en el integrado PCF8574T [5].



Fig. 4 frente y dorso del display LCD

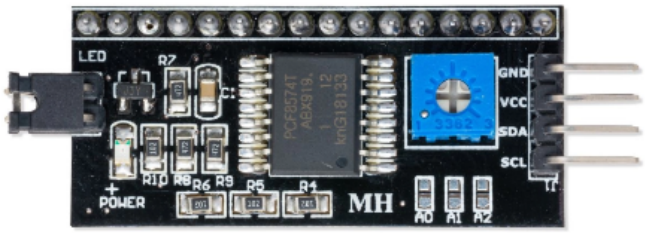


Fig. 5 adaptador basado en PCF8574T

D. Encoder rotativo

Como elemento interactivo de la HMI en el prototipo se usará un encoder rotativo ky-040 [6], este al ser presionado cambia la posición del cursor que indica qué parámetro se quiere modificar mientras que girarlo hará que la ganancia de la banda seleccionada por el cursor suba o baje dependiendo de si se giró en sentido horario o antihorario respectivamente.

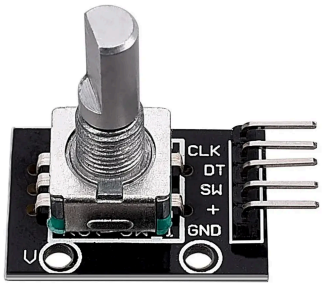


Fig. 6 encoder rotativo ky-040

E. UART PC

Para configuración remota de la ganancia de las bandas se podrá comunicar una PC por medio de protocolo UART al microcontrolador. Esta comunicación se hará por un puerto COM de la PC de forma cableada con un módulo conversor de USB a UART basado en el integrado PL2303 [7].

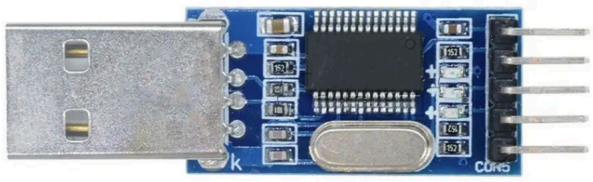


Fig. 7 adaptador basado en PL2303

F. ESP 32

Como centro de todo el diagrama estará el microcontrolador ESP32. Este se encargará de gestionar todos los periféricos del prototipo y del procesamiento de señales, todo por medio de un programa que hace uso de FreeRTOS

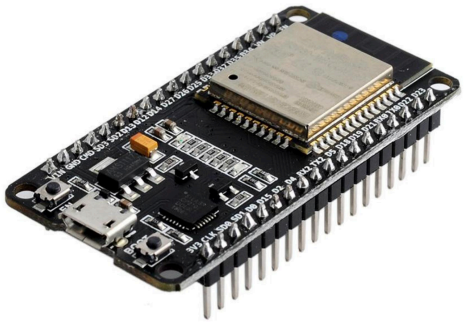


Fig. 8 placa desarrollo ESP32

VI. SOFTWARE

Desde el punto de vista del software el microcontrolador correrá un sistema operativo en tiempo real, más específicamente FreeRTOS provisto por ESPRESIF, el programa estará compuesto por un total de 6 tareas con 4 niveles de prioridad distribuidas entre los 2 núcleos del microcontrolador.

Primero determinaremos el funcionamiento del procesamiento y luego hablaremos propiamente de las tareas. Para hacer el procesamiento de señal el sistema consiste de un triple buffer rotativo, en simultáneo suceden 3 cosas: un buffer acumula las muestras entregadas por el ADC mediante el DMA del periférico I^2S; un buffer es leído y procesado por algoritmos DSP con el núcleo del ESP32 apoyándonos en la biblioteca ESP-DSP de Espressif, que proporciona funciones optimizadas para el núcleo Xtensa del ESP32, el contenido del buffer será reemplazado con la información procesada; y el último buffer envía los datos procesados al DAC mediante el DMA del periférico. Finalmente cada vez que el ADC llena un buffer y a su vez el DAC vacía el suyo (por tener frecuencias y tamaño de palabra coherentes), los buffer rotan, pasando el lleno del ADC a ser procesado, el procesado del DSP a salir por el DAC y el vaciado del DAC a ser llenado por el ADC.

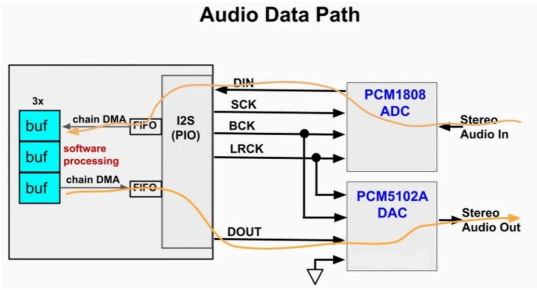


Fig. 9 datapath audio

Ahora si hablando de las tareas tenemos:

- Task_Guardian_I2S (prioridad alta, core 0): Es la responsable exclusiva de las operaciones de lectura/escritura sobre el periférico I2S y de la rotación del triple buffer. Se despierta mediante una notificación desde la ISR de recepción del DMA. Tras leer el buffer del ADC y escribir el buffer procesado al DAC, espera a que la tarea DSP termine de procesar el buffer anterior mediante el semáforo dsp_done_sem, y luego rota los punteros. Como sincronización para despertar usa un task notification del DMA, usado por su alta velocidad frente a un semáforo [8].
- Task_DSP (prioridad media, core 0): su finalidad será procesar los datos provistos del buffer del ADC por medio de filtros FIR propios de la librería ESP_DSP de espressiff, estas tienen operaciones optimizadas para el nucleo extensa del ESP [9][10][11]. Cada filtro tendrá sus coeficientes calculados por medio de una funcion dentro del programa que usa la fórmula y método provisto en la nota de aplicación de analog devices para implementación digital de un filtro Windowed-Sinc [12]. Esta tarea a la salida de los filtros le aplicará la ganancia correspondiente a la configuración, la cual leerá con un QueuePeek de la cola de acceso común “param config queue” para finalmente ser recombinados en una sola. La sincronización de esta tarea será por un xQueueReceive, la cola proviene de la tarea guardiana del I^2S y cuyo contenido es un puntero al buffer donde están los datos que envió el ADC y le entregará un semáforo binario a la tarea guardiana para indicarle que finalizó de procesar los datos.
- Task_LCD (prioridad baja, core 1): Actualiza la pantalla LCD 20×4 cada 250 ms. Lee la configuración actual de param_config_queue con xQueuePeek y muestra los valores de ganancia de las tres bandas, junto con un cursor (<) que indica cuál está seleccionada. La comunicación I2C se realiza con la API estándar de ESP-IDF. Esta tarea se sincroniza con un vTaskDelayUntill() que le da ese tiempo de 250ms de refresco.
- Task_Encoder (prioridad alta, core 1): Utiliza el módulo PCNT (Pulse Counter) provisto por espresif para el ESP32 para decodificar los pulsos de un encoder rotativo de tipo Gray. El contador se configura para incrementar o decrementar según la fase de las señales CLK y DT, eliminando la necesidad de ISRs por cada flanco. Cada vez que se acumulan 4 pulsos (ajustable), se modifica la ganancia de la banda seleccionada (en pasos de 1 dB) y se escribe el nuevo estado en param_config_queue mediante xQueueOverwrite. El pulsador (SW) se lee por polling con un antirrebote de 200 ms y cambia la banda activa. Tiene un bloqueo cada 10ms con un vTaskDelay.
- Task_UART (prioridad media, core 1): Instala el driver UART y procesa comandos enviados desde una terminal. Los comandos tienen el formato: L<valor>, M<valor> o H<valor> para ajustar la ganancia de cada banda en base a lo recibido realizará las modificaciones indicadas en “param config queue” mediante xQueueOverwrite. La sincronización de esta tarea será por medio de un xQueueReceive de una cola llamada “UART_event_queue” que contendrá la información enviada desde la terminal y en base a ello actuará. Esta tarea también entregará el semáforo llamado “change_semaphore” que indica que se hizo un cambio en la cola “param config queue”.
- Task_Flash (prioridad baja, core 1): su finalidad será tomar los parámetros actuales de “param config queue” y escribirlos en la memoria FLASH del microcontrolador para darle permanencia a la configuración entre apagados. La sincronización de esta tarea será por medio de un xSemaphoreTake de un semáforo llamado “change_semaphore”, este lo entregan tareas que hayan modificar los parámetros de “param config queue”.

VII. CONCLUSIONES

Se ha desarrollado un prototipo funcional de ecualizador digital que no solo cumple con los objetivos planteados, sino que los supera en algunos aspectos. El sistema es capaz de procesar audio en tiempo real a una frecuencia de muestreo de 44.1 kHz con una resolución de 24 bits, dividiendo la señal en tres bandas (baja, media y alta) y permitiendo un ajuste de ganancia en un rango de ±24 dB, muy por encima de los ±10 dB requeridos inicialmente. Esta ampliación fue posible gracias al amplio rango dinámico del conversor ADC/DAC y a un cuidadoso diseño del escalado y la saturación en el procesamiento.

Las mediciones con osciloscopio confirmaron que los filtros se comportan según lo esperado: las bandas baja y alta muestran una atenuación adecuada en sus frecuencias de rechazo, y la banda media presenta una respuesta plana en su rango de paso. Sin embargo, se observó un ligero ringing en la respuesta del filtro pasa-altos cuando la frecuencia de la señal de prueba se aproxima a la frecuencia de corte superior (20 kHz). Este fenómeno, inherente al diseño de filtros FIR, es aceptable para la mayoría de aplicaciones de audio, pero podría mitigarse incrementando el número de coeficientes o aplicando un sample rate mayor a costa de mayor carga computacional.

El principal inconveniente detectado fue un clipeo ocasional en la salida, que se atribuye a un problema de diseño del PCB: la distribución de las masas no proporciona un aislamiento suficiente contra interferencias electromagnéticas (EMI). Este efecto se manifiesta como un “chasquido”. Una revisión del layout, con un plano de tierra sólido y una separación adecuada entre las secciones analógica y digital, debería resolver este problema.

La interfaz de usuario, compuesta por el display LCD y el encoder rotativo, resultó intuitiva y fácil de usar. El cursor en pantalla indica claramente la banda seleccionada, y los cambios de ganancia se reflejan instantáneamente. Además, el control remoto a través de UART permite una configuración ágil desde una PC, lo que añade versatilidad al sistema.

REFERENCIAS

[1]. ESP 32
https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp32-devkitc/index.html

[2]. FreeRTOS (IDF)
https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/freertos_idf.html

[3]. PCM1808
https://www.alldatasheet.com/datasheet-pdf/view/2176993/TI2/PCM1808.html

[4]. PCM5102
https://www.alldatasheet.com/datasheet-pdf/view/841405/TI1/PCM5102.html

[5]. PCF8574T
https://www.alldatasheet.com/datasheet-pdf/view/18215/PHILIPS/PCF8574T.html

[6]. Encoder ky-040
https://www.rcscomponents.kiev.ua/datasheets/ky-040-datasheet.pdf

[7]. PL2303
https://www.alldatasheet.com/datasheet-pdf/view/1242559/PROLIFIC/PL2303.html

[8]. FreeRTOS direct to task notifications
https://www.freertos.org/Documentation/02-Kernel/02-Kernel-features/03-Direct-to-task-notifications/01-Task-notifications

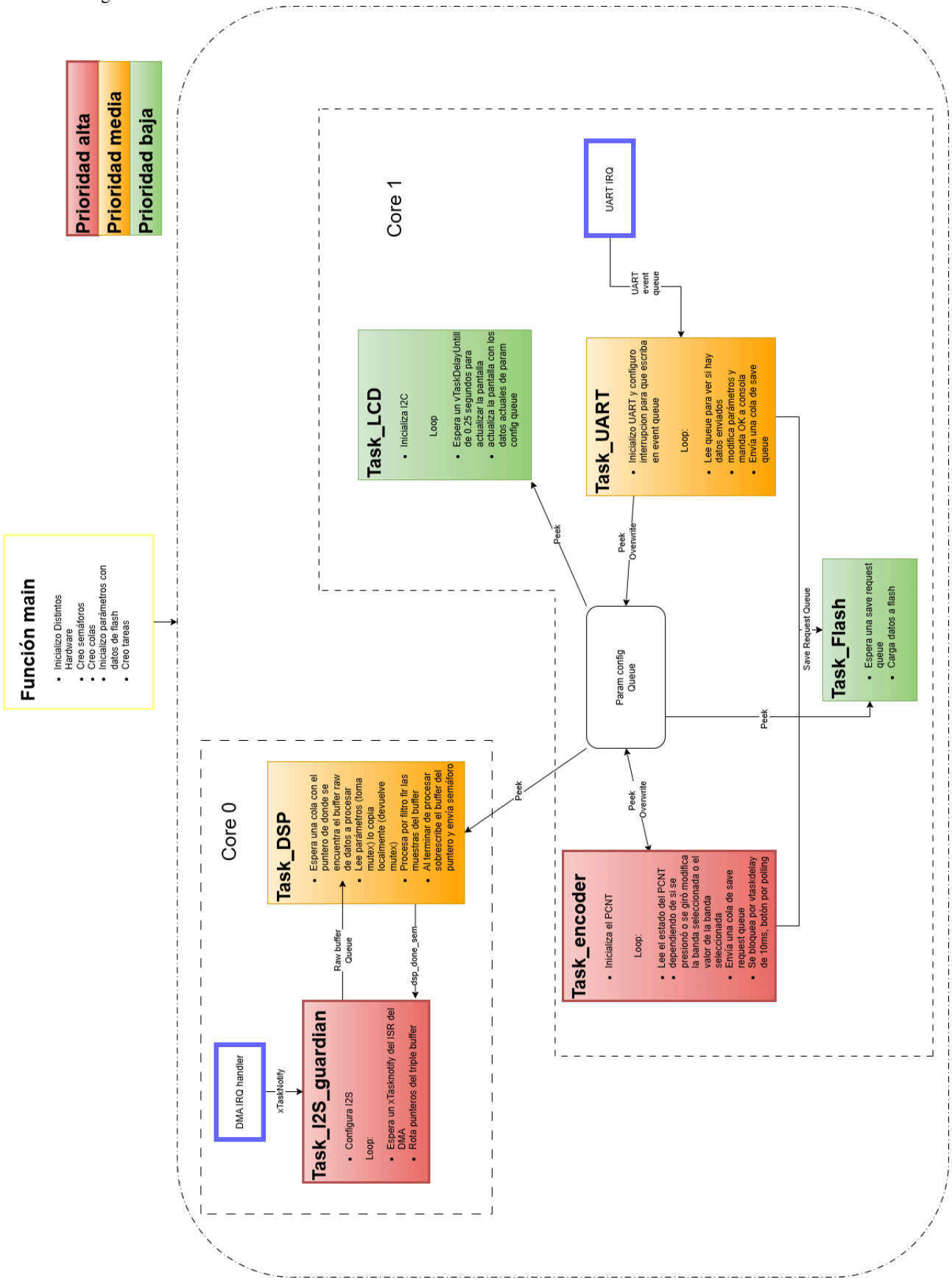
[9]. Aligned atribute
https://gcc.gnu.org/onlinedocs/gcc-4.3.2//gcc/Variable-Attributes.html

[10]. Ejemplo espresif filtro FIR
https://github.com/espressif/esp-dsp/blob/3c12d059f96ec95ed00769fb32d41ca23e20cdf1/examples/fir/main/dsps_fir_main.c

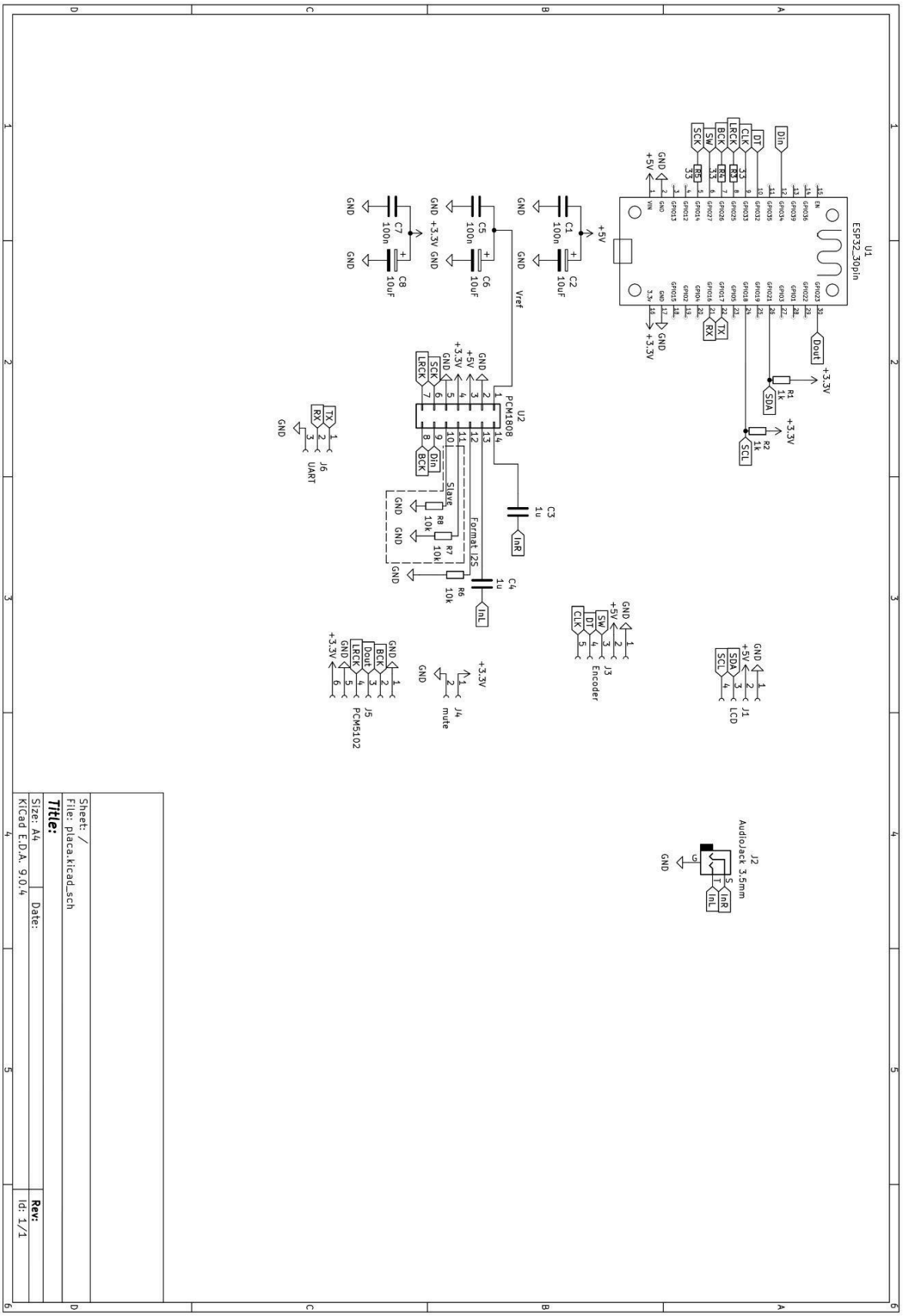
[11]. Issue github oficial espresif sobre velocidad DSP
https://github.com/espressif/esp-dsp/issues/97

[12]. Windowed sinc coeficientes
https://www.analog.com/media/en/technical-documentation/dsp-book/dsp_book_Ch16.pdf

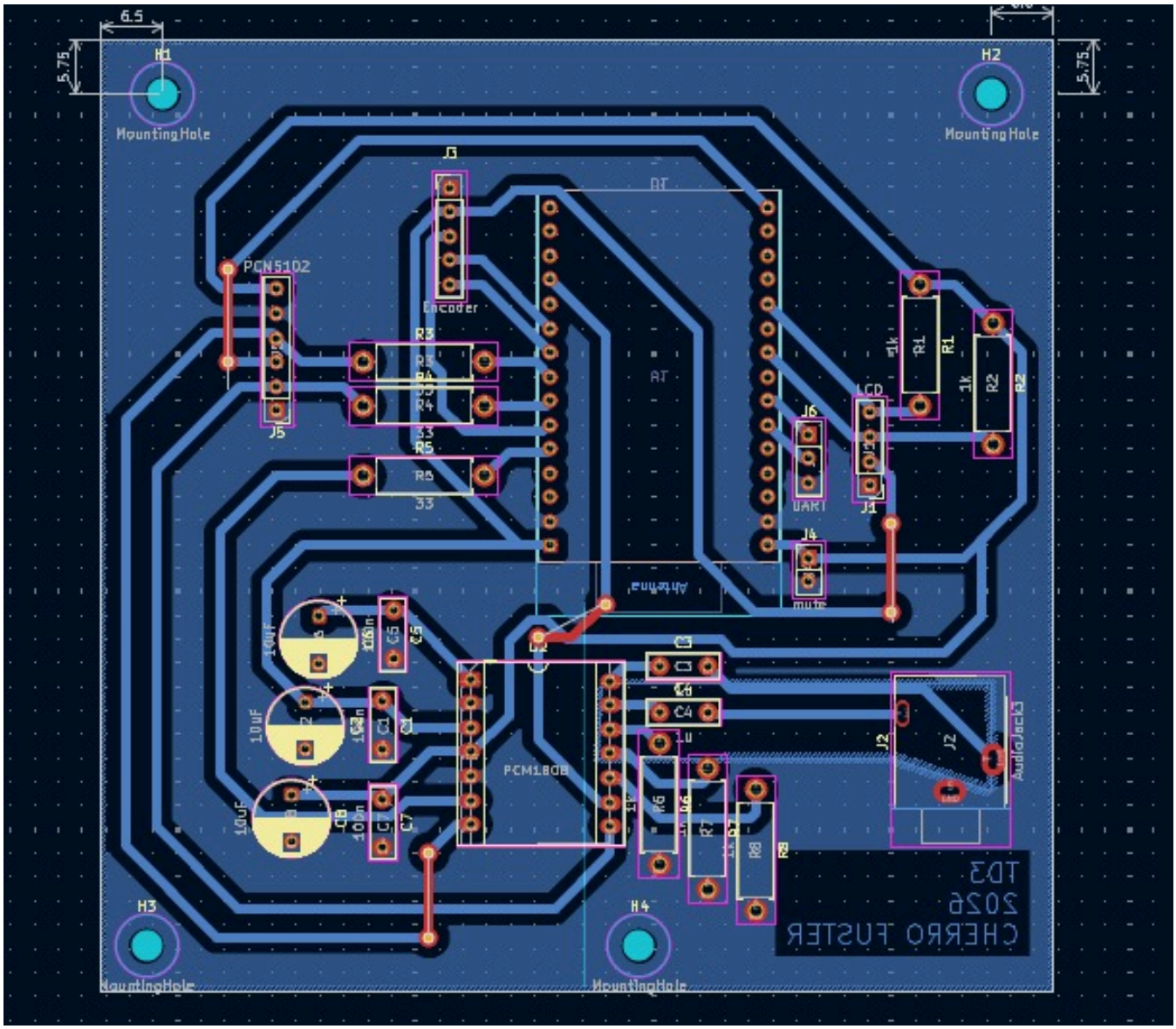
ANEXO 1 diagrama de tareas



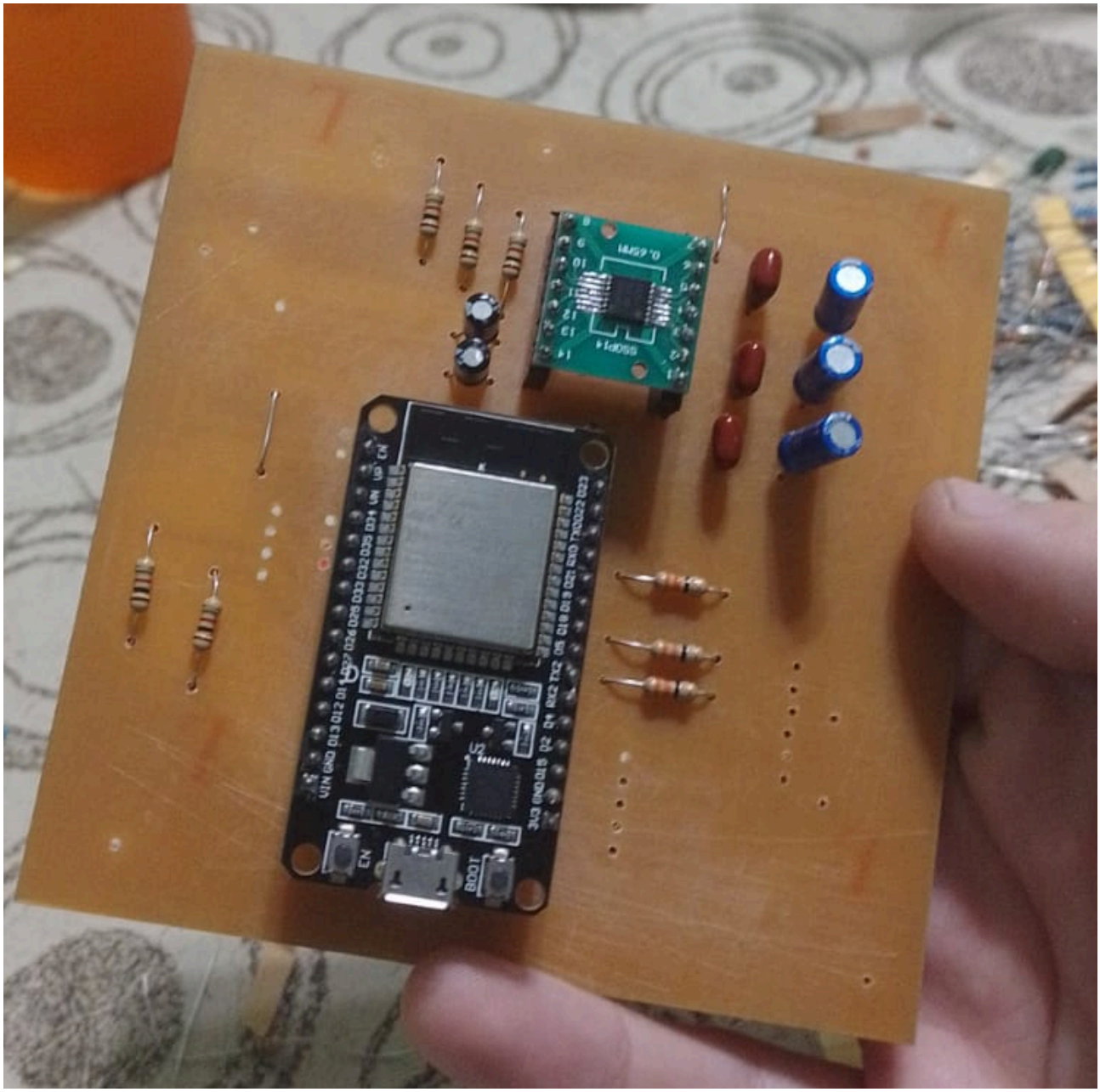
ANEXO 2 esquemático



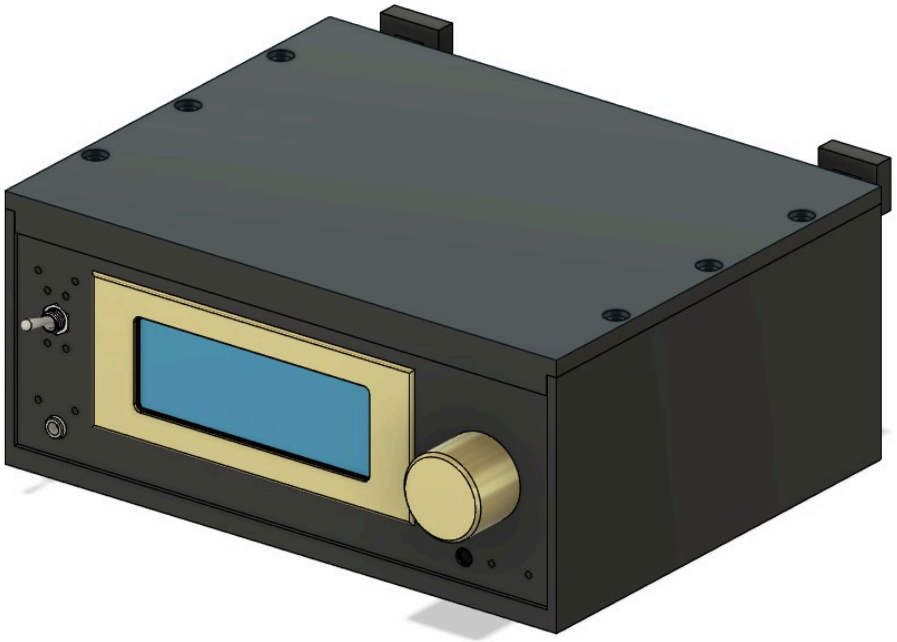
ANEXO 3 PCB layout

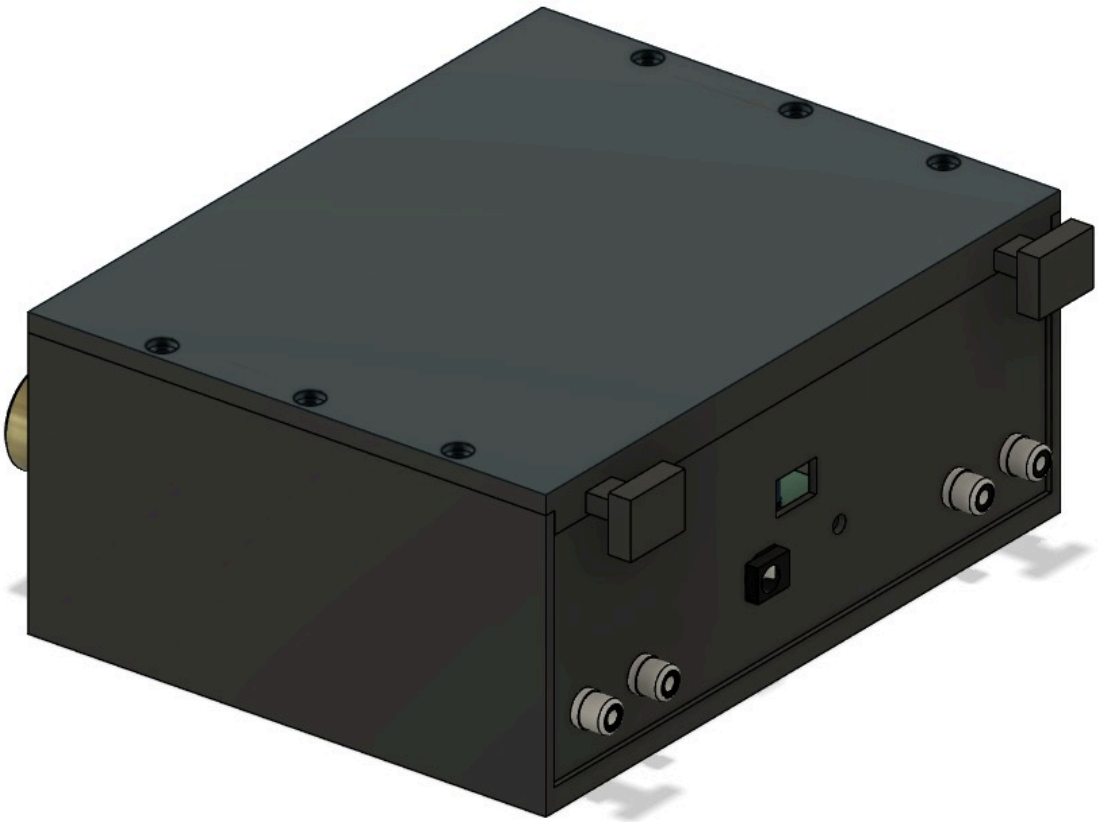


Anexo 4 PCB ensamblado

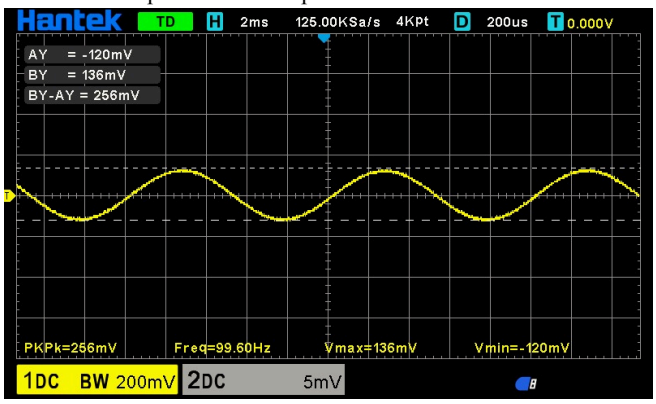


Anexo 5 renders 3D carcasa dorso y frente

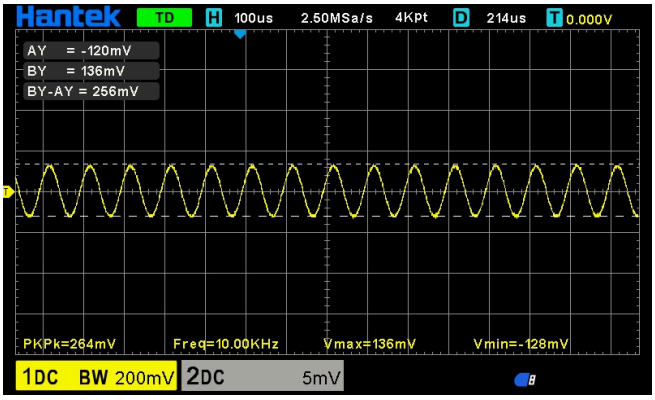




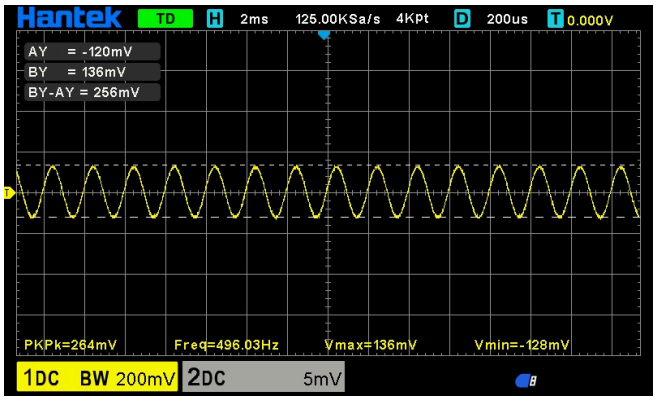
Anexo 6 capturas osciloscopio



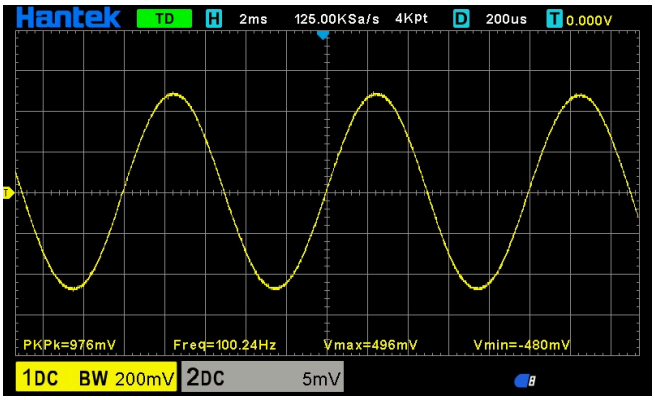
100Hz 0dB



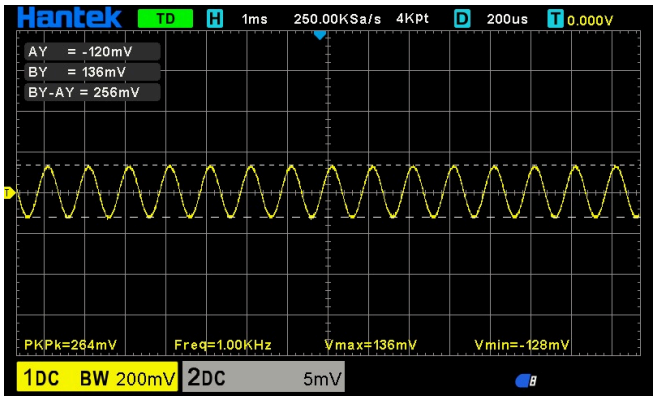
10kHz 0dB



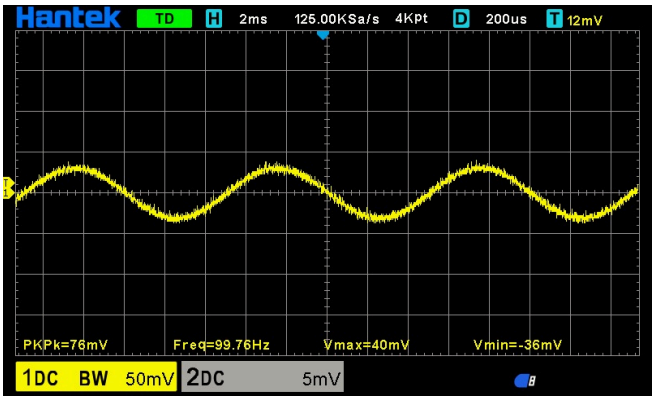
500Hz 0dB



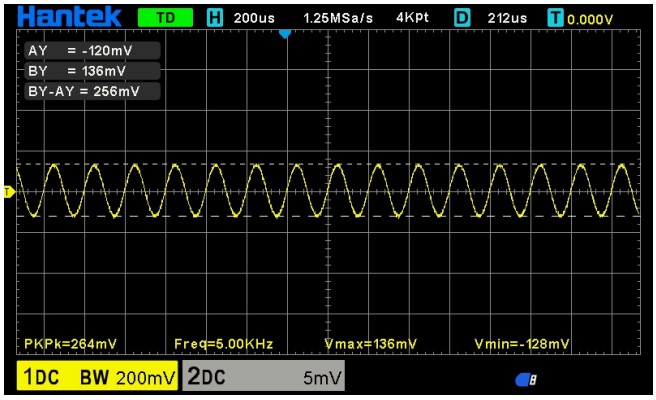
100Hz +12dB



1kHz 0dB



100Hz -12dB



5kHz 0dB

Anexo 7 ensamble final



